

00_REQUIREMENTS_CHECKLIST

Requirements Checklist — The Lenny Growth Assistant

Extracted from `Forward_Deployed_Engineer_Take_Home_Assignment.docx`. Nothing here is invented — every row traces to a section of the uploaded assignment (cited in brackets).

Status legend: NOT STARTED (Stage A only has been done so far) — this file will be updated as we implement each stage.

Product

ID	Requirement	Mandatory	Where implemented	Verification	Status
P1	PRD with user, problem, JTBD, pain points [§2]	Yes	PRD.md	Manual read	NOT STARTED
P2	≥1 measurable success metric [§2]	Yes	PRD.md	Manual read	NOT STARTED
P3	Documented assumptions (brief was incomplete) [§2]	Yes	PRD.md	Manual read	NOT STARTED
P4	Explicit scope in / scope out [§2]	Yes	PRD.md	Manual read	NOT STARTED
P5	Risks & trade-offs: hallucination, latency, cost, local-model quality, data leakage, unsafe artifact rendering [§2]	Yes	PRD.md	Manual read	NOT STARTED
P6	User flows + acceptance criteria [§6]	Yes	PRD.md	Manual read	NOT STARTED

Backend / API

ID	Requirement	Mandatory	Where implemented	Verification	Status
B1	Backend built on FastAPI [§3.1]	Yes	backend/	uvicorn runs, /docs loads	NOT STARTED
B2	New chat / session start, independent context per session [§3.1]	Yes	backend/app/routers/sessions.py	Test: two sessions don't leak context	NOT STARTED
B3	Health endpoint(s) [§3.1]	Yes	/health	curl returns 200 + component status	NOT STARTED
B4	Request/response contracts, validation, structured errors [§3.1]	Yes	Pydantic models across routers	Invalid payload → 422 with structured body	NOT STARTED
B5	Sensible HTTP status codes,	Yes (implied by §5, §8)	routers + middleware	Log inspection	NOT STARTED

	logging per endpoint				
--	----------------------	--	--	--	--

AI / Agent

ID	Requirement	Mandatory	Where implemented	Verification	Status
A1	Agent layer via Anthropic Claude Agent SDK or Pi Coding Agent [§3.1]	Yes (one of the two)	agents/	Code review + a live agent call	NOT STARTED
A2	Clear skill boundaries — not one giant prompt [§4.2 implies this generally, §7 phase language reinforces it]	Yes	skills/ modules	Code review	NOT STARTED
A3	Agent routing logic (which skill/tool handles a request) [§8 eval criteria: “reliable routing”]	Yes	agents/router.py	Test: correct skill picked per intent	NOT STARTED

Knowledge / RAG

ID	Requirement	Mandatory	Where implemented	Verification	Status
K1	Use Lenny’s Podcast/Newsletter transcript repo as data source [§3.3]	Yes	ingestion/	Ingestion log shows source count	NOT STARTED
K2	Explain ingestion: load, chunk/select, index, refresh, trace to source [§3.3]	Yes	architecture.md + ingestion/ code	Manual read + code match	NOT STARTED
K3	Answers cite / identify the transcript source used [§3.3, §4.1]	Yes	retrieval + chat response schema	Test: response includes source field	NOT STARTED
K4	Handle follow-ups, preserve session context [§4.1]	Yes	session + retrieval layer	Test: 2-turn conversation	NOT STARTED
K5	Acknowledge when material doesn’t support an answer (no hallucination) [§4.1]	Yes	prompt + retrieval-empty branch	Test: out-of-scope question	NOT STARTED

Frontend / UI

ID	Requirement	Mandatory	Where implemented	Verification	Status
F1	Chat interface, new chat/session, (conversation history where appropriate) [§3.1, own working rules]	Yes	frontend/	Manual UI test	NOT STARTED
F2	Model/provider indicator visible in UI or config [§3.2]	Yes	UI header/badge OR documented config	Manual check	NOT STARTED

F3	Grounded source references shown to user [§3.3, §4.1]	Yes	chat message component	Manual check	NOT STARTED
F4	Artifact Viewer beside/below chat, renders Markdown/HTML natively (not raw code, not external redirect) [§4.3]	Yes	frontend/components/ArtifactViewer	Manual check	NOT STARTED
F5	Loading / error / empty states	Yes (implied by §5 resilience + §8 UI/UX criteria)	UI components	Manual check	NOT STARTED
F6	Responsive, accessible	Yes (§8 eval criteria)	CSS + ally attributes	Manual check + basic audit	NOT STARTED

Artifact Generation

ID	Requirement	Mandatory	Where implemented	Verification	Status
G1	Generate Markdown documents on request [§4.3]	Yes	artifact skill/tool	Manual test	NOT STARTED
G2	Generate complete HTML/CSS snippets on request [§4.3]	Yes	artifact skill/tool	Manual test	NOT STARTED
G3	Ship 30 for 30 essay skill: ~1,250 words, strong hook, narrative progression, headings/bullets/bold, specific takeaway, grounded in transcripts, encoded as dedicated skill [§4.2]	Yes	skills/ship30_essay.py	Word count + manual read + source check	NOT STARTED

Security

ID	Requirement	Mandatory	Where implemented	Verification	Status
S1	Treat generated HTML as untrusted [§4.3 box]	Yes	artifact renderer	Code review	NOT STARTED
S2	Implement and explain isolation/sanitization strategy [§4.3 box]	Yes	architecture.md §security + ArtifactViewer (iframe sandbox)	Test: script injection attempt blocked	NOT STARTED
S3	No committed secrets [§6 deliverable 1, §5]	Yes	.gitignore , .env.example	git log -p secret scan	NOT STARTED
S4	Explain what's permitted/blocked and why [§4.3 box]	Yes	architecture.md	Manual read	NOT STARTED

Database

ID	Requirement	Mandatory	Where implemented	Verification	Status
D1	PostgreSQL persistence for	Yes	backend/app/db/	Query DB after a chat	NOT STARTED

	conversations, session IDs, timestamps, user metadata [§3.1]				
D2	May use Supabase or Railway [§3.1]	No (optional hosting)	<code>.env.example</code> / README	Manual read	NOT STARTED
D3	Graceful handling of DB connection failure [§5]	Yes	connection layer try/except + circuit breaker	Test: DB down → app still responds	NOT STARTED

Configuration

ID	Requirement	Mandatory	Where implemented	Verification	Status
C1	Provider switch without code change [§3.2]	Yes	LLMProvider abstraction + env var	Test: swap env var, restart, verify	NOT STARTED
C2	<code>.env.example</code> with safe defaults, required/optional marked [§5]	Yes	<code>.env.example</code>	Manual read	NOT STARTED
C3	Document fallback behavior between providers [§3.2]	Yes	<code>architecture.md</code> / README	Manual read	NOT STARTED

Deployment

ID	Requirement	Mandatory	Where implemented	Verification	Status
DP1	One-command startup, ideally Docker Compose [§5]	Yes	<code>docker-compose.yml</code>	<code>docker compose up --build</code>	NOT STARTED
DP2	Local Ollama demo-capable [§3.2, §6 deliverable 8]	Yes	provider config + README steps	Manual (needs your machine)	NOT STARTED
DP3	Public GitHub repository, sensible structure, no secrets [§6 deliverable 1]	Yes	GitHub (you push)	Repo URL loads, secret scan clean	BLOCKED — requires you to push

Observability

ID	Requirement	Mandatory	Where implemented	Verification	Status
O1	Structured logs for model/retrieval/DB/artifact-rendering failures [§5]	Yes	logging config	Trigger each failure, inspect logs	NOT STARTED
O2	Enough visibility for troubleshooting handoff [§5]	Yes	README §troubleshooting	Manual read	NOT STARTED

Resilience

ID	Requirement	Mandatory	Where implemented	Verification	Status
R1	Missing API keys handled gracefully [§5]	Yes	provider init	Test: unset key, app still boots	NOT STARTED
R2	Ollama unavailable handled [§5]	Yes	OllamaProvider try/except	Test: Ollama stopped	NOT STARTED
R3	Model timeout	Yes	provider timeout	Test: forced	NOT STARTED

	handled [§5]		wrapper	timeout	
R4	Empty retrieval result handled [§5]	Yes	retrieval branch	Test: obscure query	NOT STARTED
R5	DB connection failure handled [§5]	Yes	DB layer	Test: DB down	NOT STARTED

Testing

ID	Requirement	Mandatory	Where implemented	Verification	Status
T1	Automated tests: API, retrieval, routing, persistence [§6 deliverable 7]	Yes	tests/	pytest run + real assertions	NOT STARTED
T2	Manual UI test plan [§6 deliverable 7]	Yes	docs/manual_test_plan.md	Manual read	NOT STARTED

Documentation

ID	Requirement	Mandatory	Where implemented	Verification	Status
DOC1	README.md: architecture overview, prerequisites, install, env vars, local/cloud model setup, run commands, tests, troubleshooting [§6 deliverable 2]	Yes	README.md	Fresh-clone read-through	NOT STARTED
DOC2	PRD (see Product section) [§6 deliverable 3]	Yes	PRD.md	Manual read	NOT STARTED
DOC3	design.md: UI/UX principles, IA, key states, responsive, accessibility, design decisions [§6 deliverable 4]	Yes	design.md	Manual read	NOT STARTED
DOC4	architecture.md: DB schema, API endpoints, component boundaries, ingestion/retrieval flow, agent routing, model toggle, security, deployment topology [§6 deliverable 5]	Yes	architecture.md	Manual read	NOT STARTED

GitHub / Agent Transcripts

ID	Requirement	Mandatory	Where implemented	Verification	Status
GH1	Coding-agent transcripts/logs in dedicated folder, including failed attempts + corrections, secrets	Yes	agent-transcripts/	Manual read	NOT STARTED

	stripped [\$6 deliverable 6]				
GH2	Public repo pushed to GitHub [\$6 deliverable 1]	Yes	GitHub	Repo loads publicly	BLOCKED — requires you to push

Demo Video

ID	Requirement	Mandatory	Where implemented	Verification	Status
V1	2-3 min video, camera on, explain problem, show product, demonstrate local Ollama, cover one trade-off, uploaded to YouTube [\$6 deliverable 8]	Yes	You record it	Watch the upload	BLOCKED — requires you to record

Submission

ID	Requirement	Mandatory	Where implemented	Verification	Status
SUB1	Submit via https://forms.gle/LgotDHNvXW1mbzNE7 [\$7]	Yes	You submit	Confirmation	BLOCKED — requires you; also due date (15/09/26) has passed as of today 16/09/26 — confirm with the employer before assuming this is still valid

What I flagged as ambiguous → assumptions I'll make (to document in the PRD)

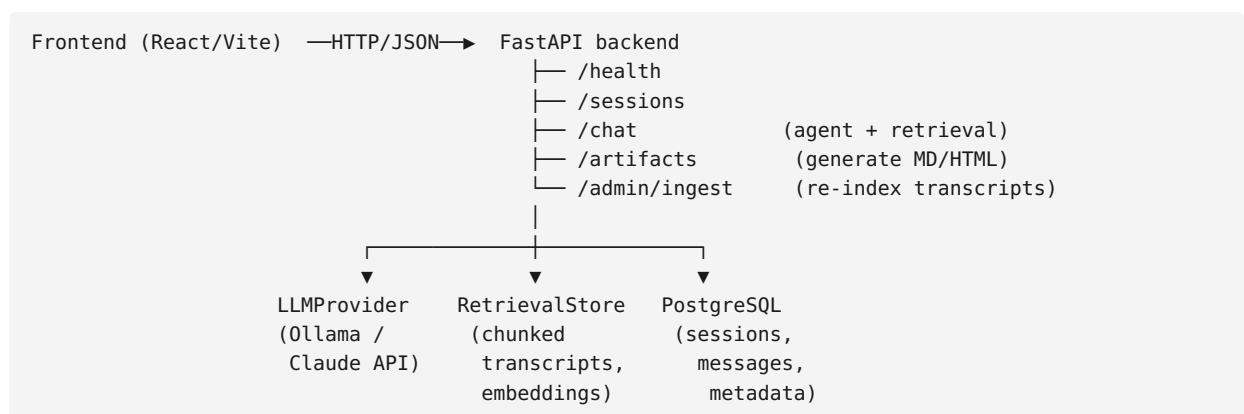
1. **Agent SDK choice:** assignment allows Claude Agent SDK or Pi Coding Agent. I'll default to the **Claude Agent SDK / a lightweight Claude tool-use agent pattern**, since it's the better-documented option I can verify against real API behavior — document this as an explicit PRD assumption.
2. **Cloud LLM provider:** I'll default to **Anthropic Claude** for the cloud provider (consistent with the agent layer), with the provider abstraction designed so OpenAI could be swapped in.
3. **Transcript ingestion:** I cannot `git clone github.com/ChatPRD/lennys-podcast-transcripts` from this sandbox (outside network allowlist). I will build the ingestion pipeline against that repo's expected structure and give you the exact one-line command to run locally; if the repo layout differs from what I assume, the ingestion script will need a small adjustment on your machine — I'll document that clearly rather than pretend I ingested real data I never saw.
4. **Hosting for Postgres:** I'll target a **local Postgres via Docker Compose** as the default (works with one command, no external account needed), and document Supabase/Railway as optional alternatives per the assignment's "may use."
5. **"Conversation history where appropriate":** I'll implement a session list/sidebar so users can return to prior sessions — reasonable for a product like this even though not explicitly mandatory.

External resources requiring action on your machine (I cannot do these from this sandbox)

--	--	--

Resource	Why I can't do it here	What you'll run
Clone transcript repo	Network allowlist doesn't include raw GitHub content hosts for arbitrary repos in this exact form / large data pull	<code>git clone https://github.com/ChatPRD/lennys-podcast-transcripts transcripts/</code>
Install & run Ollama	No local model runtime in this container	Install from ollama.com, <code>ollama pull <model></code> (I'll recommend one sized for a laptop)
Push to GitHub	I have no GitHub write access / credentials	<code>git init && git remote add origin ... && git push</code>
Record & upload demo video	No camera/recording capability	Follow the demo script I'll produce in Stage J
Submit the form	Requires your identity/authorization	Fill out the Google Form link

Recommended Architecture (summary — full detail goes in `architecture.md` at Stage H)



- **Provider abstraction:** `LLMProvider` base class → `OllamaProvider`, `ClaudeProvider`; selected via `MODEL_PROVIDER` env var, with fallback order documented.
- **Agent layer:** router determines intent (grounded Q&A vs. Ship 30 essay vs. artifact generation) and dispatches to the matching skill module.
- **Retrieval:** chunk transcripts → embed → store (start with a simple local vector store to avoid extra infra; documented as swappable for pgvector/Supabase vector).
- **Artifact security:** HTML artifacts rendered in a **sandboxed** `<iframe>` with `sandbox="allow-scripts"` deliberately **omitted** by default (no script execution), `srcdoc` injection instead of same-origin render, and a strict allowlist documented in `architecture.md`.

Recommended Repo Structure

```

lenny-growth-assistant/
├── backend/
├── frontend/
├── agents/
├── skills/
├── knowledge/
├── ingestion/
├── tests/
├── docs/
├── transcripts/      # gitignored raw data; ingestion writes derived index here
├── scripts/
├── docker/
├── agent-transcripts/
├── PRD.md
├── README.md
├── design.md
├── architecture.md
├── .env.example
├── docker-compose.yml
└── FINAL_SUBMISSION_CHECKLIST.md  # written at Stage J
  
```

Implementation Plan (Stages B-J, per your instructions)

Stage	Deliverable	Depends on
B	Backend skeleton: FastAPI app, health, sessions, DB models/migrations	A (this doc)
C	Ingestion + retrieval (chunking, index, source tracing)	B
D	Agent layer + skills (grounded Q&A, Ship 30 essay, artifact generation)	B, C
E	Frontend: chat UI, Artifact Viewer, provider indicator	B, D
F	Docker Compose, .env.example, config/fallback logic	B-E
G	Automated tests + manual test plan	B-F
H	PRD.md, design.md, architecture.md, README.md	all above
I	Evaluator audit — fresh-environment run-through, fix issues	H
J	FINAL_SUBMISSION_CHECKLIST.md + demo script	I

Key Risks / Trade-offs (expanded in PRD)

- **Hallucination** if retrieval is weak or the prompt doesn't hard-constrain the model to cite sources.
- **Local model quality** (Ollama) may be noticeably weaker than cloud Claude for the Ship 30 essay task — I'll pick a model and note this trade-off explicitly rather than overclaim quality.
- **Latency** from RAG + agent routing + cloud calls — will document expected response times.
- **No vector DB infra by default** — trading retrieval sophistication for one-command reproducibility; documented as a scope choice, with pgvector noted as an upgrade path.
- **I cannot verify the transcript repo's exact file format from this sandbox** — ingestion code will be written defensively and you'll confirm it against the real repo on first run.